

PCU-Select: Amortizing Target-Specific Data Selection Across PEFT Configurations

Anonymous submission

Abstract

A large language model (LLM) serving system typically maintains a registry of fine-tuning configurations for diverse tasks and deployment budgets. These configurations vary in which modules are trained and how they are updated. This diversity poses a challenge to gradient-based data selection: the selector must rebuild a target-specific gradient datastore for every configuration, making cost scale linearly with registry size. Reusing one subset avoids this cost but can reduce quality because different configurations favor different examples. PCU-Select reuses expensive selection computations while producing a distinct subset for each PEFT–task pair. The key insight is to map different PEFT methods onto a common set of 24 transformer outputs through which PEFT updates affect the backbone. We cache reusable sample and task features at these outputs and combine them with a compact encoding of the target configuration. After a one-time offline stage, a single scorer ranks candidates for any supported PEFT–task pair without rebuilding gradients. In our experiments, PCU-Select reduces the cost of onboarding a new configuration by $42\times$ compared to LESS, from 63.2 to 1.51 GPU-hours, while maintaining downstream quality within a negligible margin of the baseline. By amortizing per-configuration computation into a one-time offline stage, PCU-Select makes gradient-level data selection practical at registry scale.

Introduction

Large language model platforms must adapt one backbone to different tasks and resource limits. Parameter-efficient fine-tuning (PEFT) reduces adaptation cost by updating only a small set of model parameters, allowing a platform to maintain a *registry* of configurations that differ in method, trainable modules, capacity, or training recipe. These trainable parameters determine a configuration’s *trainable subspace*: the update directions available during fine-tuning. Each training example can affect the model only through the directions that the target configuration exposes. Data selection reduces cost along a complementary axis by choosing a compact subset whose examples produce useful task updates. Its decisions therefore depend on the target trainable subspace: changing

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

the subspace can change which examples the selector should prioritize.

Most data selectors ignore this interaction. Quality filters, diversity heuristics, and proxy-model scores produce one ranking for a task and reuse it across PEFT configurations. We call such a ranking *PEFT-agnostic* because the selector does not receive the target configuration. Yet common PEFT methods permit different updates: LoRA adds low-rank changes to selected projections (Hu et al. 2022); adapters insert trainable bottlenecks (Houlsby et al. 2019); and (IA)³ scales selected activations (Liu et al. 2022). The same example can therefore produce different updates, and hence have different value, under different configurations.

Gradient-based selectors capture this dependence but incur repeated computation. LESS (Xia et al. 2024) first builds a target-specific *gradient datastore*, a cache containing a training gradient for each candidate example. It then ranks the candidates against a *task sketch*, a small set of training or development examples that represents the target task. Because each PEFT configuration exposes different trainable parameters, LESS must rebuild the cache for every configuration. Its selection cost therefore grows linearly with the registry.

One shortcut is to reuse a single LESS subset for every configuration. This reuse avoids repeated computation but removes the configuration information that makes gradient selection useful. In our evaluation, the shortcut performs worse than a PEFT-agnostic representation baseline (Table 2). Existing methods thus force a choice: recompute a target-specific ranking for every configuration or reuse a ranking that ignores the configuration.

The configuration dependence affects downstream performance as well as the rankings. Figure 1 shows that independent runs for the same configuration agree more closely than runs across configurations. Changes within one PEFT family preserve more of the ranking, while cross-family changes produce the largest disagreement. Matched selection improves performance by 2.0 points over cross-family reuse and by 1.45 points within LoRA configurations (Supplementary Table S10 and Figure S1).

These observations motivate *PEFT-conditioned utility*: the value of training on example x for task t under PEFT configuration p . We denote this value by $u(x, p, t)$. A task-only selector omits p and must assign the same ranking to every

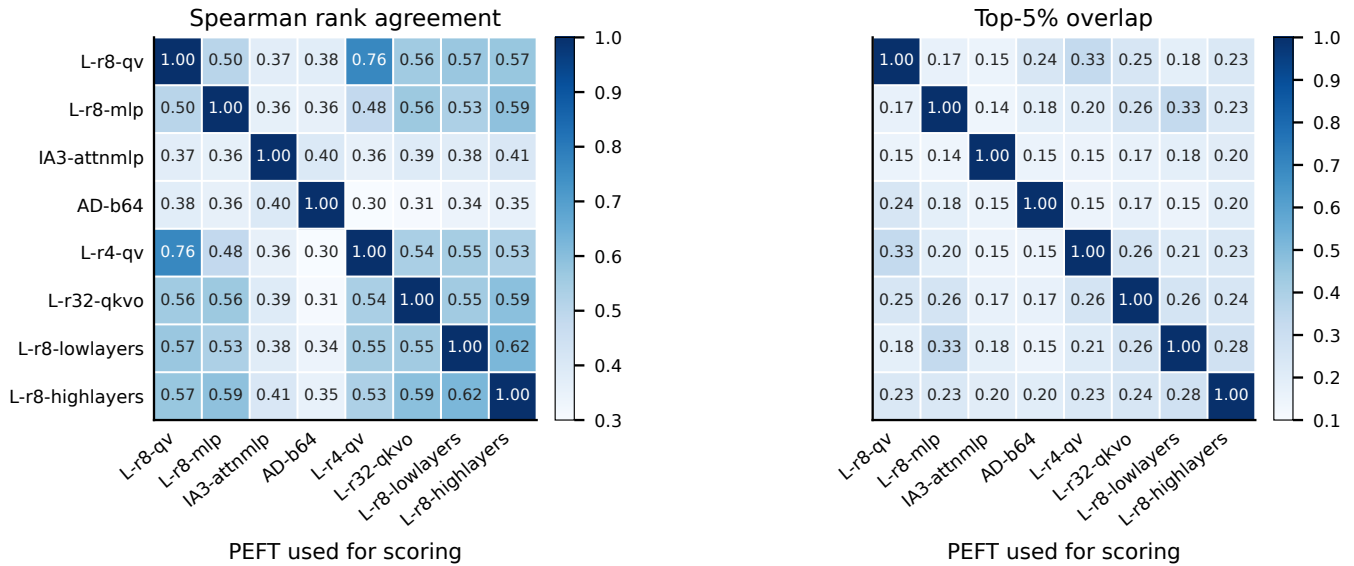


Figure 1: Left: Spearman rank agreement between PEFT-conditioned utility rankings. Right: Top-5% overlap between the corresponding selected examples; lower off-diagonal values indicate stronger configuration dependence.

configuration. Per-PEFT influence selection includes p but recomputes gradients for each target. Our goal is to predict $u(x, p, t)$ while sharing the expensive computations used to learn these scores.

PCU-Select (PEFT-Conditional Utility) enables this sharing through a common representation. It maps every supported configuration to 24 *intervention sites*: selected Transformer module outputs through which PEFT updates affect the backbone. These outputs remain comparable even when PEFT methods train parameter tensors with different shapes. PCU-Select caches sample and task signals at the sites. A structured PEFT code records which sites a configuration affects, its update type, its capacity, and its training recipe. Supplying this code at scoring time preserves configuration information while keeping the cached signals reusable.

A conditional scorer turns the shared signals into a target-specific ranking. Its prediction depends on the candidate example, task sketch, and PEFT code. It learns from two complementary signals: inexpensive labels based on gradient alignment cover the full candidate pool, while short-update labels measure task-sketch loss reductions for a smaller sample. At deployment, the scorer ranks the cached examples for a target configuration and task. PCU-Select then spreads the budget across groups of semantically similar examples to limit near-duplicates. This pipeline moves gradient extraction and label construction offline while retaining a distinct subset for each target.

These design choices amortize target-specific selection across a PEFT registry. We make three contributions:

- **PEFT-conditioned utility.** We define data value as a candidate example’s usefulness for a given task and PEFT configuration. We show that both rankings and downstream subset quality change with the target configuration.

- **Intervention-site amortization.** PCU-Select maps different PEFT parameterizations to 24 shared module-output sites. A structured code retains each target’s update structure and recipe, enabling configuration-specific rankings without rebuilding gradient datastores; ablations validate both PEFT and task conditioning.
- **Registry-scale quality and cost.** Across four tasks and five seen PEFTs on Llama-2-7B (Touvron et al. 2023), PCU-Select gains 0.74 points over RDS+ and matches per-PEFT LESS within ± 0.04 , while cutting four-task PEFT onboarding by $42\times$, from 63.2 to 1.51 GPU-hours.

Related Work

Instruction-data selection. Instruction-data selectors usually estimate quality or diversity independently of the target PEFT configuration. LIMA supports instruction tuning with 1,000 curated examples (Zhou et al. 2023). Model-based filters instead use judgments or instruction properties: Alpaca filters Alpaca examples with LLM judgments (Chen et al. 2024), InsTag measures semantic diversity and complexity through instruction tags (Lu et al. 2023), and DEITA combines quality, complexity, and diversity (Liu et al. 2024). Other methods select for explicit diversity through quality-diversity trade-offs, log-determinant objectives, or partition-aware sampling (Bukharin et al. 2023; Wang et al. 2024; Rao et al. 2024). These methods improve data efficiency, but their rankings omit the target PEFT’s trainable parameters.

Proxy-model and influence selection. Proxy-model selectors provide reusable rankings without computing gradients for every target PEFT. Cherry LLM uses instruction-following difficulty (Li et al. 2023); Superfiltering studies when weak models can rank data for stronger models (Li et al. 2024); and SmallToLarge transfers training-trajectory information from small models to larger targets (Yang et al.

2024). Because these scores encode only example and task information, a PEFT change leaves the ranking unchanged.

Influence-based selectors model training effects more directly, but their gradients tie the score to a target parameterization. Classical influence functions use second-order approximations (Koh and Liang 2017), while TracIn accumulates gradient similarities across checkpoints (Pruthi et al. 2020). LESS builds a low-dimensional gradient datastore for instruction selection (Xia et al. 2024), and DataInf specializes influence approximation for LoRA-tuned generative models (Kwon et al. 2024). Preserving configuration specificity therefore requires rebuilding PEFT-dependent gradients when the target changes. PCU-Select instead conditions one scorer on a structured PEFT code, combining reusable sample signals with a target-specific ranking. Our evaluation compares PCU-Select with both a per-PEFT LESS implementation and an Influence baseline that uses one shared datastore.

Parameter-efficient fine-tuning. PEFT methods expose different trainable subspaces through bottlenecks, low-rank updates, activation or normalization scaling, and continuous controls (Houlsby et al. 2019; Pfeiffer et al. 2021; Hu et al. 2022; Dettmers et al. 2023; Liu et al. 2022; Wang et al. 2022; Li and Liang 2021; Lester, Al-Rfou, and Constant 2021). Prior work compares their accuracy and efficiency with fixed training data; we test whether configurations induce different rankings over one candidate pool.

Joint data- and parameter-efficient adaptation. The closest methods use PEFT gradients for data selection, but their utility estimates remain specific to one target configuration. DataInf and LESS compute selection signals from LoRA-style training (Kwon et al. 2024; Xia et al. 2024). Geometry-aware targeted selection further shows that useful data depends on the directions available to PEFT optimization (Min et al. 2026). These methods must regenerate configuration-dependent signals when the target changes. PCU-Select instead describes each configuration by its intervention sites, update type, capacity, and training recipe. This representation lets one scorer reuse sample and task signals while still producing a different ranking for each configuration.

Problem Formulation

We study *cross-PEFT data-efficient fine-tuning*: selecting a small training subset for a target task and PEFT configuration while reusing selection work across configurations. We fix a backbone family and a frozen base model M_0 . The candidate pool $\mathcal{D} = \{x_i\}_{i=1}^N$ contains N instruction–response examples. Each task t provides a *task sketch* V_t , a small set from its training or development data. Each PEFT configuration $p \in \mathcal{P}$ specifies a method, trainable locations, capacity, and training recipe. For a selection budget B , the ideal subset maximizes held-out task performance:

$$\mathcal{S}^* = \arg \max_{\mathcal{S} \subseteq \mathcal{D}, |\mathcal{S}| \leq B} \text{Perf}(\text{Tune}(M_0, p, \mathcal{S}), V_t^{\text{test}}). \quad (1)$$

Here Tune adapts M_0 with configuration p and subset $\mathcal{S}$, and Perf evaluates the adapted model. The held-out test set

V_t^{test} defines this ideal objective but never enters training, scoring, or selection; those procedures use only V_t . Directly solving Eq. (1) would require training and evaluating many subsets for every target (p, t) pair.

PCU-Select avoids this search by assigning each example a conditional utility: its benefit for task t when training uses configuration p . Its mean prediction approximates this utility:

$$\hat{\mu}_\phi(x, p, t) \approx u(x, p, t). \quad (2)$$

The target $u(x, p, t) \in [0, 1]$ summarizes task-sketch loss reductions after a few PEFT update steps. PCU-Select rank-normalizes these reductions within groups that share the same PEFT, task, and measurement setting (see “Method: PCU-Select”). Including p allows the ranking to change with the trainable subspace; omitting p would force every configuration to share one task-specific ranking. The task sketch V_t identifies the desired task behavior, while p identifies the updates available to the model. Utility is therefore relative to both the task and the PEFT configuration.

Amortization objective. A registry changes the cost objective because it serves multiple PEFT configurations and tasks from one candidate pool. Let P be the number of target configurations and Q the number of task sketches. LESS builds one gradient datastore for each configuration and ranks that datastore once for each task. PCU-Select instead pays one shared offline cost, followed by one scoring-and-selection pass for every PEFT–task pair:

$$\begin{aligned} C_{\text{LESS}}(P, Q) &= PC_{\text{LESS}}^{\text{data}} + PQ C_{\text{LESS}}^{\text{rank}}, \\ C_{\text{PCU}}(P, Q) &= C_{\text{offline}} + PQ C_{\text{PCU}}^{\text{pair}}. \end{aligned} \quad (3)$$

Equation (3) separates four costs. $C_{\text{LESS}}^{\text{data}}$ builds one PEFT-specific datastore, and $C_{\text{LESS}}^{\text{rank}}$ ranks it for one task. C_{offline} extracts reusable signals and trains PCU-Select once. $C_{\text{PCU}}^{\text{pair}}$ scores and selects data for one target pair. LESS builds P datastores and performs PQ rankings, whereas PCU-Select pays the offline cost once. The onboarding-cost experiment measures each cost term.

The objective is to retain per-PEFT LESS quality while replacing repeated datastore construction with shared offline work. Solving the two cost equations gives a break-even registry size; the onboarding-cost experiment substitutes the measured costs.

Method: PCU-Select

PCU-Select learns the conditional utility surrogate in Eq. (2) and selects a budgeted subset for a target PEFT–task pair. Figure 2 summarizes two stages. Offline, it extracts reusable sample and task signals, constructs two types of utility label, and trains one scorer. Online, it combines cached sample features with a target (p^*, t^*) , scores the pool, and distributes budget across semantic clusters. Both stages use deterministic cluster quotas and the same cached representations.

Shared Intervention Sites Enable Cross-PEFT Reuse

PEFT families expose parameter tensors with different shapes and roles, so their raw gradients cannot share one

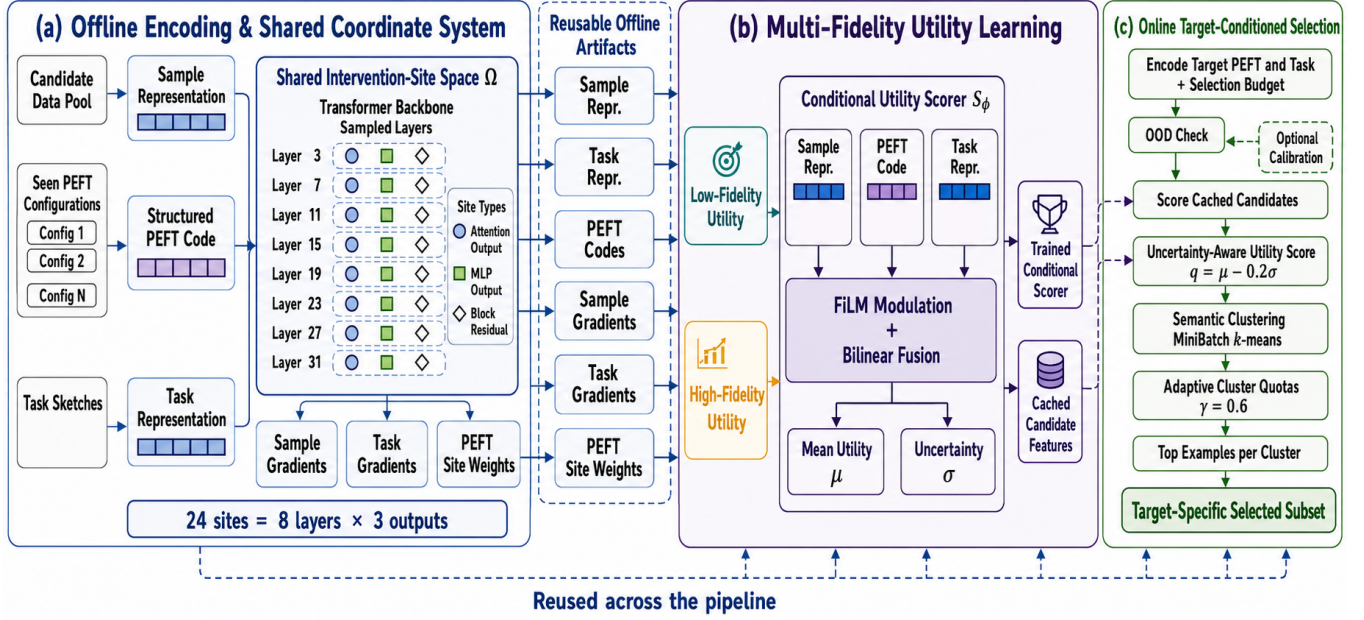


Figure 2: PCU-Select learns from reusable site-level signals and scores each PEFT–task target online. A 24-site coordinate system aligns sample and task gradients with a 128-dimensional structured PEFT code; proxy and short-update labels provide complementary supervision.

cache (Hu et al. 2022; Houlisby et al. 2019; Liu et al. 2022). PCU-Select instead uses a fixed set Ω of *intervention sites*. Each site is a selected module output through which a PEFT update affects the backbone. The set Ω is a shared coordinate system because sample gradients, task gradients, and each PEFT’s map of affected sites all use the same index ω .

PCU-Select hooks attention, MLP, and post-residual outputs at eight evenly spaced Transformer layers, giving $|\Omega| = 24$. A site’s response-loss gradient describes how a sample or task pushes that activation. By the chain rule, PEFT-parameter gradients factor through gradients at the module outputs they modify. Site gradients therefore avoid both full PEFT-parameter reconstruction and shape mismatches between fused and split projections. Post-residual sites place adapters in the same index space as LoRA and (IA)³.

A site index records only *where* a configuration acts. PCU-Select separately encodes operator and capacity features to record *how* it acts and how much it can update. This factorization reuses one sample signature without treating additive, multiplicative, and bottleneck updates as equivalent.

Each configuration maps to a site mask and operator type. Attention LoRA and (IA)³-attention map to attention-output sites. MLP LoRA and (IA)³-FFN map to MLP-output sites, while adapters map to block-residual sites. Prefix tuning maps to attention-output sites. LN Tuning updates Llama-2’s per-block RMSNorm scales (Wang et al. 2022; Touvron et al. 2023): input-normalization scales map to attention-output sites, and post-attention-normalization scales map to MLP-output sites. The configuration excludes the final model-level RMSNorm. Both site types use the multiplicative operator

channel. These mappings define each normalized site weight:

$$\begin{aligned} w_p^\omega &= \text{mask}(p, \omega) \kappa(p, \omega), \\ \kappa(p, \omega) &= \tanh(\eta \log(1 + \text{cap}(p, \omega)/d_{\text{model}})), \\ \tilde{w}_p^\omega &= w_p^\omega / \max(\varepsilon_w, \sum_{\omega'} w_p^{\omega'}). \end{aligned} \quad (4)$$

Here $\text{mask}(p, \omega)$ is zero at untouched sites, and $\text{cap}(p, \omega)$ measures trainable capacity. The width d_{model} normalizes capacity, η controls saturation through $\tanh$, and ε_w keeps the denominator nonzero. Thus, $\tilde{w}_p^\omega$ distributes unit weight over the sites used by p . Capacity corresponds to LoRA rank, adapter width, (IA)³ vector count, prefix length, or LN-Tuning scale count. Operator type remains separate in z_p because a family-wide scalar would cancel under Eq. (4)’s normalization.

Shared Representations Preserve Target Specificity

Shared site indices make signals comparable, but the scorer must still distinguish each sample, task, and PEFT. PCU-Select uses three fixed-length vectors: the sample representation describes the example, the task representation describes the target need, and the PEFT representation describes where and how training acts. Site gradients supply separate label-construction signals.

The sample representation $z_x = [e_x; d_x; a_x] \in \mathbb{R}^{848}$ combines semantic, descriptive, and activation features. The 768-dimensional e_x is the frozen sentence-transformers/all-mpnet-base-v2 embedding of the instruction and response (Song et al. 2020; Reimers and Gurevych 2019). This encoder remains

frozen during pool and target-benchmark processing. The 16-dimensional d_x records length, response-LM loss, entropy, and format. The 64-dimensional a_x summarizes response-token hidden states at the eight sampled layers.

Utility is task-relative, so PCU-Select forms $z_t \in \mathbb{R}^{848}$ by mean-pooling z_x over the task sketch’s 32 examples. For proxy labels, it separately averages their site gradients into g_t^ω . These 24×256 values remain outside z_t and serve only label construction.

The PEFT representation records the target trainable subspace. In $z_p = [m_p; c_p; r_p] \in \mathbb{R}^{128}$, the 24×4 mask m_p contributes 96 dimensions. Its channels mark active sites, additive updates, multiplicative scaling, and prefix-style control. Two 16-dimensional vectors encode capacity c_p and training recipe r_p . At this site resolution, q/v and q/k/v/o LoRA share attention-output sites, while capacity and recipe preserve their projection-level distinction.

Prefix, P-Tuning, and LN Tuning use the same deterministic encoding rules as seen families. The scorer has no learned family lookup or operator centroid. The support policy below decides whether a target uses direct scoring, calibrated scoring, or a per-target selector.

Training these representations requires labels for many (x, p, t) triples, but measuring every PEFT update would defeat amortization. PCU-Select instead uses *multi-fidelity supervision*, which combines labels with different costs and directness. A low-fidelity gradient proxy cheaply covers the pool. High-fidelity short updates directly measure sketch-loss reduction for fewer triples. Together, they learn broad ranking structure and correct errors near the subset cutoff.

The *gradient-proxy label* measures alignment between a sample and the task sketch at sites used by the target PEFT:

$$u^{\text{lo}}(x, p, t) = \sum_{\omega \in \Omega} \tilde{w}_p^\omega \cos(g_x^\omega, g_t^\omega). \quad (5)$$

Here g_x^ω is the response-token loss gradient at site ω . A Johnson–Lindenstrauss random projection (Johnson and Lindenstrauss 1984) compresses it to 256 dimensions and unit-normalizes the result. The task-sketch mean gives g_t^ω . Equation (5) therefore measures alignment only at sites used by p . All sketches use response-token cross-entropy for these signatures; task-native metrics such as option likelihood, Pass@1, and F1 remain downstream evaluation metrics. Because cosine values can be negative, PCU-Select rank-normalizes them within each (p, t) bucket before proxy distillation and ranking. Cached sample signatures then label many (x, p, t) triples without new gradient extraction.

The *short-update label* measures realized sketch-loss reduction near the subset cutoff. For each sampled triple, PCU-Select uses two anchors θ_a : the base model and a warm checkpoint after adaptation has begun. The horizon h is the number of short-update steps. A mini-batch $\mathcal{B}(x)$ contains the sampled example and seven condition-matched fillers. PCU-Select applies PEFT p for h steps and measures

$$\Delta(x, p, t, a, h) = L_{V_t}(\theta_a) - L_{V_t}(\text{Adapt}_p^h(\mathcal{B}(x); \theta_a)). \quad (6)$$

Thus, Δ is the task-sketch loss before the update minus its loss afterward. Rank-normalized reductions for $h \in \{1, 4\}$

from both anchors define u^{hi} . The fillers make this mini-batch effect more stable than a single-example update. PCU-Select allocates 10K short-update labels in three stages. Coverage sampling first spreads labels across the registry without a scorer.

Coverage labels and the proxy then train a provisional scorer. It predicts ranks and a per-triple uncertainty scale, an estimate of prediction noise. Uncertainty sampling selects triples with large scales. Boundary sampling selects large predicted–observed rank disagreements near the cutoff. Every rank normalization compares labels within one PEFT, task, anchor, and horizon bucket.

The proxy cheaply orders the full pool, while short updates target consequential ranking errors. Two anchors test whether utility persists after adaptation begins; two horizons reduce dependence on one update length. Ranking within condition-matched buckets avoids assuming a shared raw-loss scale across tasks or configurations.

Conditional Scoring Produces Target-Specific Subsets

Given (z_x, z_p, z_t) , the scorer s_ϕ outputs $(\hat{\mu}_\phi, \hat{\sigma}_\phi)$, where $\hat{\mu}_\phi$ estimates utility and $\hat{\sigma}_\phi$ estimates a positive, input-dependent uncertainty scale. Separate encoders map the three inputs into hidden vectors. FiLM modulation (Perez et al. 2018) uses the PEFT and task codes to rescale and shift sample features. A bilinear term represents sample–PEFT compatibility.

Stage A distills gradient-proxy ranks to learn broad conditional structure. Stage B continues proxy distillation and adds ranking and regression on short-update labels plus a heteroscedastic uncertainty loss. Here *heteroscedastic* means that predicted noise can differ across triples. PCU-Select selects the Stage B checkpoint on held-out PEFT–task pairs. It also fixes the sketch size, label allocation, anchors, horizons, and scalar selection hyperparameters on these pairs before downstream evaluation.

A target’s structural support level determines whether PCU-Select applies the scorer directly or calibrates it. Relative to the seen registry, L0 resembles a seen configuration, L1 makes a larger shift within a seen family, and L2 introduces an unseen family. Seen targets use the trained scorer directly. L0 targets run *zero-shot*: PCU-Select scores a new configuration without target-specific short-update labels. L1 and L2 targets use calibrated scoring when compatible labels exist and a per-target selector otherwise.

After any required calibration, PCU-Select converts the two scorer outputs into a conservative ranking score:

$$q(x) = \hat{\mu}_\phi(x, p^*, t^*) - \lambda \hat{\sigma}_\phi(x, p^*, t^*), \quad \lambda = 0.2. \quad (7)$$

The uncertainty scale reduces the rank of examples whose predicted utility is noisy. It serves as a ranking penalty, not a confidence interval.

Ranking by q alone can over-select near-duplicates, so PCU-Select clusters examples in semantic space before assigning the budget. It allocates budget B across K clusters C_k , where $k \in \{1, \dots, K\}$:

$$b_k = \text{round} \left(B \frac{(v_k^+)^{\gamma} |C_k|^{1-\gamma}}{\sum_j (v_j^+)^{\gamma} |C_j|^{1-\gamma}} \right), \quad \gamma = 0.6. \quad (8)$$

Here v_k is the mean conservative score among the top 10% of examples in C_k , and $v_k^+ = \max(v_k, 0)$. The factors v_k^+ and $|C_k|$ balance cluster value and size. For the 300K pool, $K = \max(50, \lfloor \sqrt{N} \rfloor)$ produces 547 clusters of about 548 examples. The reported end-to-end cost is 0.38 GPU-hours per PEFT-task application and includes clustering. PCU-Select takes the top- b_k examples by q within each cluster. Non-positive clusters receive no budget; deterministic remainder handling makes the quotas sum to B .

For seen and L0 targets, all gradient extraction and label construction occur offline; online selection uses cached features and scorer forward passes. L1 and calibratable L2 targets add a small label set without rebuilding a target-specific gradient datastore. The evaluation next tests whether this reuse preserves LESS-level quality, reduces registry cost, and transfers across support levels.

Experiments

The evaluation tests whether PCU-Select preserves per-PEFT LESS’s average downstream quality, reduces the marginal cost of adding a PEFT, and transfers to new configurations according to their structural support in the training registry. We then identify which components sustain the reusable scorer.

Setup

We evaluate Llama-2-7B (Touvron et al. 2023) on GSM8K, HumanEval, MMLU, and TyDiQA (Cobbe et al. 2021; Chen et al. 2021; Hendrycks et al. 2021; Clark et al. 2020), using exact match, Pass@1, accuracy, and F1, respectively. We report percentages and their equal-weight macro-average as a registry-level summary. Per-task results appear in Supplementary Table S1. Every selector chooses 10% from a decontaminated pool of 300K mixed instruction examples. Each task sketch contains 32 fixed-seed examples from the task’s training or development split and remains disjoint from evaluation. HumanEval uses MBPP and CodeAlpaca because it lacks a training split; content hashes and AST-normalized signatures remove all sketch examples from the candidate pool. The pool combines seven public instruction corpora after exact, normalized-string, 13-gram, and MinHash decontamination; each selected subset contains 30K examples.

The main comparison covers five configurations seen during scorer training: attention and MLP LoRA, (IA)³, and adapters. The transfer study holds out configurations that vary rank, placement, learning rate, or family, including Prefix Tuning, P-Tuning, and LN Tuning (Wang et al. 2022). We instantiate LN Tuning by updating the scale parameters of both per-block RMSNorm modules and excluding the final model-level RMSNorm (Touvron et al. 2023). Supplementary Table S3 lists all 17 configurations.

The four baselines differ in target specificity. **Random** uses neither task nor PEFT information. **RDS+**, our representation-similarity baseline, ranks frozen semantic embeddings by similarity to the task-sketch mean but ignores the PEFT configuration. **Influence**, our shared-datastore gradient baseline, ranks projected base-model gradients from one PEFT-agnostic datastore. **LESS**, our per-PEFT implementation of gradient-influence selection (Xia et al.

2024), warms up each target adapter, builds an Adam-preconditioned gradient datastore, and ranks examples by sketch influence. PCU-Select uses its conditional scorer, uncertainty penalty, and adaptive semantic-cluster quotas.

We hold the backbone, PEFT recipe, training steps, and evaluation protocol constant, so only the selected subset changes. Target adaptation trains only PEFT parameters for 1000 AdamW steps with a cosine schedule, global batch 128, and maximum sequence length 1024. All costs use eight NVIDIA H20-96GB GPUs and the wall-time-times-GPU-count convention. The main table varies target-training seeds 0, 1, and 2 while fixing the selector, scorer, sketch, and pool at seed 0. Matched seeds give every selector the same training randomness. We obtain each descriptive 95% interval by resampling the evaluated PEFT-task cells under this fixed selection pipeline.

PCU-Select Near-Ties Per-PEFT LESS

Table 1 shows that PCU-Select improves on the PEFT-agnostic baselines and remains close to target-specific LESS; Supplementary Table S1 gives task-wise aggregates and win counts for the 20 PEFT×task cells. It gains 0.74 points over RDS+ (17/20 wins; interval $[+0.38, +1.06]$) and 0.50 over Influence (18/20; $[+0.21, +0.75]$). Against LESS, its mean difference is +0.04 points ($[-0.26, +0.33]$; 10/20 wins).

Performance varies across tasks and PEFTs, so an average near-tie is the appropriately scoped registry-level conclusion. The task-grouped interval $[-0.15, +0.22]$ also crosses zero.

Amortization Cuts Four-Task PEFT Onboarding Cost by 42×

Table 2 separates *fixed/shared* cost, paid once, from *marginal* cost, paid for each new PEFT. All selector costs exclude the shared 12.8 GPU-hour downstream run. PCU-Select and per-PEFT LESS have near-tied average quality but distribute selection cost differently. LESS has no fixed cost and spends 63.2 GPU-hours per four-task configuration. PCU-Select pays 144.0 GPU-hours once, then 1.51 GPU-hours per new configuration: a 42× lower marginal cost.

The gap comes from configuration reuse. Adding a task to an existing PEFT costs one online-selection pass: 0.38 GPU-hours for PCU-Select and 0.30 for LESS. The PCU-Select value rounds the exact average per-task cost of 0.3775. Adding a PEFT forces LESS to rebuild its datastore, whereas PCU-Select reuses the scorer (Supplementary Table S5). The cost benefit therefore comes from cross-PEFT reuse.

The fixed-subset shortcut trades away quality. **Reuse-one-LESS** applies one LESS subset to all five targets; it costs 63.2 GPU-hours but averages 34.24, below RDS+ at 34.44. Reusing a subset selected for one trainable subspace removes the target condition. PCU-Select produces a distinct subset for each target and retains LESS-level average quality.

The measured costs break even at 2.33 four-task configurations. Serving the five-PEFT registry costs 151.6 GPU-hours with PCU-Select and 316.0 with LESS, a 2.09× reduction. For targets far from the training registry, optional calibration uses 500 *reduced calibration labels* from one warm anchor and one update step. This procedure costs 1.05 GPU-hours per (p, t) , one quarter of the ≈ 4.2 GPU-hours for the

Method	AD-b64	IA3-attnmlp	L-r16-qkvo	L-r8-mlp	L-r8-qv	Avg.
Random	32.79±0.38	31.74±0.13	33.11±0.23	31.97±0.21	31.82±0.16	32.29±0.16
RDS+	35.27±0.23	33.54±0.26	35.00±0.29	34.46±0.21	33.91±0.17	34.44±0.13
Influence	35.08±0.27	33.84±0.26	35.24±0.19	34.69±0.13	34.53±0.22	34.68±0.11
LESS	35.52±0.20	34.44±0.13	35.80±0.15	35.13±0.14	34.80±0.37	35.14±0.14
PCU-Select	35.61±0.40	34.60±0.21	35.93±0.30	34.56±0.23	35.18±0.31	35.18±0.16

Table 1: Each PEFT column reports the equal-weight macro-average of the four task-native percentage scores. Values are mean $\pm$ standard deviation across three matched downstream-training seeds; the selector, task sketch, candidate pool, and scorer are fixed. Across the 20 configuration–task cells, the paired PCU-Select-minus-LESS difference is +0.04 points, with a descriptive 95% interval of $[-0.26, +0.33]$.

Method	Fixed	Marg.	Total	Quality	Gap
Random	0.0	0.0	0.0	32.29	−2.85
RDS+	1.6	0.0	1.6	34.44	−0.70
Influence	48.8	0.0	48.8	34.68	−0.46
Reuse-one-LESS	63.2	0.0	63.2	34.24	−0.90
LESS	0.0	63.2	316.0	35.14	0.00
PCU-Select	144.0	1.51	151.6	35.18	+0.04

Table 2: Quality and selection cost across five seen PEFTs. Costs are GPU-hours and exclude downstream tuning; *Gap* is the paired difference from LESS in task-native points. Boldface marks the best quality and marginal cost.

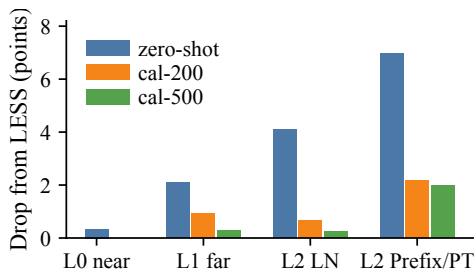


Figure 3: Mean LESS-minus-PCU-Select gap across each tier’s targets and three tasks (lower is better). Cal-200/500 denote calibration-label counts.

full routine. It shifts the worst-case break-even point to 2.50 configurations. For supported four-task configurations that require no calibration, PCU-Select’s marginal selection cost is $42\times$ lower than per-PEFT LESS.

Reduced Calibration Narrows Gaps Beyond the Training Registry

Before evaluation, we group held-out targets by structural support: L0 resembles a seen configuration, L1 makes a larger shift within a seen family, and L2 introduces an unseen family.

Cal-200/500 freeze the shared scorer and fit only a zero-initialized linear residual head using 200/500 target-specific short-update labels. Because each label uses one warm anchor and one update, the two budgets cost only 0.42 and 1.05 GPU-hours per PEFT–task pair.

Against LESS, the zero-shot gap grows from 0.34 points at L0 to 2.08 at L1, 4.08 for LN Tuning, and 6.97 for Prefix/P-Tuning (Figure 3). Cal-200 already cuts the L1 and LN-Tuning gaps to 0.91 and 0.65; Cal-500 further lowers them to 0.30 and 0.23, removing 86% and 94% of the original gaps. The additional 300 labels yield only 0.61 and 0.42 points of recovery, indicating that a lightweight residual correction recovers most of the shared scorer’s transferable ranking structure. The Prefix/P-Tuning gap falls from 6.97 in the zero-shot setting to 2.17 with Cal-200 and 1.99 with Cal-500; its 0.18-point Cal-200-to-Cal-500 change is descriptive.

Ablations Identify the Useful Components

Removing the PEFT code causes the largest downstream drop, 1.37 points; removing the task sketch costs 0.82 points. A family-only code loses 0.68 points, and uniform rather than adaptive cluster quotas loses 0.51, showing that configuration structure and value-aware coverage both matter. Using only gradient-proxy or short-update labels loses 0.85 or 0.36 points, respectively. Replacing adaptive quotas with global top- k loses 1.08 points, while removing the activation signature or uncertainty penalty costs 0.48 or 0.28 points. Supplementary Table S9 reports the full ablation on GSM8K and HumanEval with two representative PEFTs.

Limitations

Evaluation remains limited to one backbone family, four tasks, and a finite registry; broader backbones, domains, and update mechanisms require validation. Short-update labels approximate full-training contribution, and structural tiers are registry-relative. Finally, amortization is most relevant when a service repeatedly onboards configurations rather than serving a single target.

Conclusion

PCU-Select makes PEFT-conditioned selection reusable, producing target-specific subsets without rebuilding gradient datastores. Across four tasks and five seen configurations, it matches per-PEFT LESS (35.18 versus 35.14), cuts four-task onboarding by $42\times$, and lowers five-PEFT selection cost by $2.09\times$. Calibration recovers 86% of the L1 gap and 94% of the LN-Tuning gap; the PEFT-code ablation further supports the role of structural conditioning. Overall, PCU-Select combines target-specific quality, efficient transfer, and low marginal cost in a reusable selector.

References

- Bukharin, A.; Li, S.; Wang, Z.; Yang, J.; Yin, B.; Li, X.; Zhang, C.; Zhao, T.; and Jiang, H. 2023. Data Diversity Matters for Robust Instruction Tuning. *arXiv preprint arXiv:2311.14736*.
- Chen, L.; Li, S.; Yan, J.; Wang, H.; Gunaratna, K.; Yadav, V.; Tang, Z.; Srinivasan, V.; Zhou, T.; Huang, H.; and Jin, H. 2024. AlpaGasus: Training A Better Alpaca with Fewer Data. In *The Twelfth International Conference on Learning Representations*.
- Chen, M.; et al. 2021. Evaluating Large Language Models Trained on Code. *arXiv preprint arXiv:2107.03374*.
- Clark, J. H.; Choi, E.; Collins, M.; Garrette, D.; Kwiatkowski, T.; Nikolaev, V.; and Palomaki, J. 2020. TyDi QA: A Benchmark for Information-Seeking Question Answering in Typologically Diverse Languages. *Transactions of the Association for Computational Linguistics*, 8: 454–470.
- Cobbe, K.; Kosaraju, V.; Bavarian, M.; Chen, M.; Jun, H.; Kaiser, L.; Plappert, M.; Tworek, J.; Hilton, J.; Nakano, R.; Hesse, C.; and Schulman, J. 2021. Training Verifiers to Solve Math Word Problems. *arXiv preprint arXiv:2110.14168*.
- Dettmers, T.; Pagnoni, A.; Holtzman, A.; and Zettlemoyer, L. 2023. QLoRA: Efficient Finetuning of Quantized LLMs. In *Advances in Neural Information Processing Systems*, volume 36.
- Hendrycks, D.; Burns, C.; Basart, S.; Zou, A.; Mazeika, M.; Song, D.; and Steinhardt, J. 2021. Measuring Massive Multitask Language Understanding. In *International Conference on Learning Representations*.
- Houlsby, N.; Giurgiu, A.; Jastrzebski, S.; Morrone, B.; de Laroussilhe, Q.; Gesmundo, A.; Attariyan, M.; and Gelly, S. 2019. Parameter-Efficient Transfer Learning for NLP. In *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, 2790–2799. PMLR.
- Hu, E. J.; Shen, Y.; Wallis, P.; Allen-Zhu, Z.; Li, Y.; Wang, S.; Wang, L.; and Chen, W. 2022. LoRA: Low-Rank Adaptation of Large Language Models. In *International Conference on Learning Representations*.
- Johnson, W. B.; and Lindenstrauss, J. 1984. Extensions of Lipschitz Mappings into a Hilbert Space. In *Conference in Modern Analysis and Probability*, volume 26 of *Contemporary Mathematics*, 189–206. American Mathematical Society.
- Koh, P. W.; and Liang, P. 2017. Understanding Black-box Predictions via Influence Functions. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, 1885–1894. PMLR.
- Kwon, Y.; Wu, E.; Wu, K.; and Zou, J. 2024. DataInf: Efficiently Estimating Data Influence in LoRA-tuned LLMs and Diffusion Models. In *The Twelfth International Conference on Learning Representations*.
- Lester, B.; Al-Rfou, R.; and Constant, N. 2021. The Power of Scale for Parameter-Efficient Prompt Tuning. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, 3045–3059. Association for Computational Linguistics.
- Li, M.; Zhang, Y.; He, S.; Li, Z.; Zhao, H.; Wang, J.; Cheng, N.; and Zhou, T. 2024. Superfiltering: Weak-to-Strong Data Filtering for Fast Instruction-Tuning. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Bangkok, Thailand: Association for Computational Linguistics.
- Li, M.; Zhang, Y.; Li, Z.; Chen, J.; Chen, L.; Cheng, N.; Wang, J.; Zhou, T.; and Xiao, J. 2023. From Quantity to Quality: Boosting LLM Performance with Self-Guided Data Selection for Instruction Tuning. *arXiv preprint arXiv:2308.12032*.
- Li, X. L.; and Liang, P. 2021. Prefix-Tuning: Optimizing Continuous Prompts for Generation. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing*, 4582–4597. Association for Computational Linguistics.
- Liu, H.; Tam, D.; Muqeeth, M.; Mohta, J.; Huang, T.; Bansal, M.; and Raffel, C. 2022. Few-Shot Parameter-Efficient Fine-Tuning is Better and Cheaper than In-Context Learning. In *Advances in Neural Information Processing Systems*, volume 35.
- Liu, W.; Zeng, W.; He, K.; Jiang, Y.; and He, J. 2024. What Makes Good Data for Alignment? A Comprehensive Study of Automatic Data Selection in Instruction Tuning. In *The Twelfth International Conference on Learning Representations*.
- Lu, K.; Yuan, H.; Yuan, Z.; Lin, R.; Lin, J.; Tan, C.; Zhou, C.; and Zhou, J. 2023. InsTag: Instruction Tagging for Analyzing Supervised Fine-tuning of Large Language Models. *arXiv preprint arXiv:2308.07074*.
- Min, G.; Huang, T.; Wan, K.; and Chen, C. 2026. GIST: Targeted Data Selection for Instruction Tuning via Coupled Optimization Geometry. *arXiv preprint arXiv:2602.18584*.
- Perez, E.; Strub, F.; de Vries, H.; Dumoulin, V.; and Courville, A. 2018. FiLM: Visual Reasoning with a General Conditioning Layer. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 32, 3942–3951.
- Pfeiffer, J.; Kamath, A.; Rücklé, A.; Cho, K.; and Gurevych, I. 2021. AdapterFusion: Non-Destructive Task Composition for Transfer Learning. In *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics: Main Volume*, 487–503. Online: Association for Computational Linguistics.
- Pruthi, G.; Liu, F.; Sundararajan, M.; and Kale, S. 2020. Estimating Training Data Influence by Tracing Gradient Descent. In *Advances in Neural Information Processing Systems*, volume 33.
- Rao, J.; Liu, X.; Lian, L.; Cheng, S.; Liao, Y.; and Zhang, M. 2024. CommonIT: Commonality-Aware Instruction Tuning for Large Language Models via Data Partitions. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, 10064–10083. Miami, Florida, USA: Association for Computational Linguistics.

Reimers, N.; and Gurevych, I. 2019. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing*, 3982–3992. Association for Computational Linguistics.

Song, K.; Tan, X.; Qin, T.; Lu, J.; and Liu, T.-Y. 2020. MP-Net: Masked and Permuted Pre-training for Language Understanding. In *Advances in Neural Information Processing Systems*, volume 33, 16857–16867.

Touvron, H.; et al. 2023. Llama 2: Open Foundation and Fine-Tuned Chat Models. *arXiv preprint arXiv:2307.09288*.

Wang, P.; Shen, Y.; Guo, Z.; Stallone, M.; Kim, Y.; Golland, P.; and Panda, R. 2024. Diversity Measurement and Subset Selection for Instruction Tuning Datasets. *arXiv preprint arXiv:2402.02318*.

Wang, Q.; Ruan, Y.-P.; Zuo, Y.; and Li, T. 2022. Parameter-Efficient Tuning on Layer Normalization for Pre-trained Language Models. *arXiv preprint arXiv:2211.08682*.

Xia, M.; Malladi, S.; Gururangan, S.; Arora, S.; and Chen, D. 2024. LESS: Selecting Influential Data for Targeted Instruction Tuning. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, 54104–54132. PMLR.

Yang, Y.; Mishra, S.; Chiang, J. N.; and Mirzasoleiman, B. 2024. SmallToLarge (S2L): Scalable Data Selection for Fine-tuning Large Language Models by Summarizing Training Trajectories of Small Models. In *Advances in Neural Information Processing Systems*, volume 37.

Zhou, C.; Liu, P.; Xu, P.; Iyer, S.; Sun, J.; Mao, Y.; Ma, X.; Efrat, A.; Yu, P.; Yu, L.; Zhang, S.; Ghosh, G.; Lewis, M.; Zettlemoyer, L.; and Levy, O. 2023. LIMA: Less Is More for Alignment. In *Advances in Neural Information Processing Systems*, volume 36.